

SAVEY PERSONAL — PRODUCT & BUSINESS DOCUMENTATION

Version: 1.0

Product: Savey Personal

Status: In Development

Owner: Savey Africa

1. PRODUCT OVERVIEW

1.1 What is Savey

Savey is a structured financial management platform that helps individuals plan, allocate, save, spend, and track money intentionally.

Unlike traditional wallets or banking apps that only show balance, Savey ensures that every amount of money has a purpose, category, or plan before it is spent.

Savey combines:

- Wallet
- Budgeting
- Savings
- Spending control
- Reports
- Requests
- Financial planning

into one unified system.

Savey is designed for:

- Individuals
- Families
- Couples
- Friends
- Freelancers
- SMEs
- Irregular earners
- Salary earners

1.2 Founding Insight

Savey was built on the insight:

People do not fail financially because they lack money.
They fail because money arrives without structure.

Most financial apps allow users to:

- Receive money
- Spend money
- Transfer money

But they do not help users:

- Plan money
- Separate money
- Allocate money
- Control spending
- Save intentionally

Savey solves this by enforcing structure before spending.

1.3 Product Philosophy

Savey follows these core principles:

1. Money must have purpose before spending
2. Budget must exist before expense
3. Savings must be intentional
4. Wallet is not equal to available money
5. Spending must be tracked by category
6. Financial decisions must be visible
7. Users should be accountable to their plans
8. Financial structure should be automatic when possible

This philosophy affects all features including:

- Budgeting
- Spend
- Savings
- Requests
- Reports
- Funding
- Goals

1.4 What makes Savey different

Traditional apps:

- Show balance
- Allow transfer
- Allow bill payment

Savey:

- Requires budget before spend
- Supports ledger-based budgeting
- Supports automatic savings
- Supports structured requests
- Supports savings types
- Supports reports
- Supports allocation
- Supports financial planning

Savey is closer to:

- Personal finance system
- Financial OS
- Structured wallet
- Budget-driven banking

not just a fintech wallet.

1.5 Product Scope

Current product:

- Savey Personal

Future products:

- Savey Family
- Savey Business
- Savey Groups / ESUSU
- Savey AI Finance
- Savey Africa ecosystem

Savey Personal is the foundation for all others.

1.6 Current Feature Coverage (High Level)

Implemented / In progress:

- Top up (bank, card)
- Spend (budget-based)

- Request (saver / friend)
- Budgeting
- Savings (rainy day, flexible, fixed, esusu, bsg)
- Reports
- Transaction history
- Beneficiaries
- Bills (in progress)
- Scheduled payments
- Split payments

Planned:

- Loan
- Insurance
- Investment
- AI budgeting
- AI savings
- AI planning

1.7 Target Users

Primary users:

- Salary earners
- Freelancers
- Students
- Couples
- Families
- Small business owners
- People with irregular income
- People trying to save
- People trying to control spending

Secondary users (future):

- Groups
- Cooperatives
- SMEs
- Churches
- Communities

1.8 Product Goal

The goal of Savey Personal is to:

- Help users control money

- Help users save consistently
- Help users plan spending
- Help users track finances
- Help users avoid waste
- Help users reach goals
- Help users build financial discipline

Long term goal:

Become the financial structure platform for Africa.

2. CORE PRODUCT PRINCIPLES (VERY IMPORTANT)

This will define rules like:

- Budget must be selected before spend
- Ledger system
- Wallet rules
- Savings rules
- Request rules
- Funding rules

This is the foundation of the whole system.

Continuing the document.

2. CORE PRODUCT PRINCIPLES

This section defines the **rules that govern how Savey works internally**.

All features must follow these principles.

These rules separate Savey from normal banking or wallet apps.

These principles affect:

- Wallet
- Budget
- Spend
- Savings
- Requests
- Reports
- Funding
- Ledger
- Future features

2.1 Structure Before Spending

In Savey, money should not be spent without structure.

Structure means:

- Budget
- Category
- Plan
- Savings allocation
- Goal
- Purpose

Rule:

A user should not spend money without selecting a budget or category.

This rule is already implemented in Spend.

Example:

User must select budget before sending money.

2.2 Budget-Driven Spending

Savey uses a **budget-first system**, not balance-first.

Traditional apps:

Balance → Spend

Savey:

Budget → Spend → Wallet

Rules:

- Spending must belong to a budget
- Budget controls spending visibility
- Reports depend on budget
- Savings may depend on budget

This allows:

- Accurate reports
- Discipline
- Planning
- AI insights later

2.3 Ledger-Based Finance System

Savey uses multiple internal ledgers.

Not only one balance.

Possible ledgers:

- Main wallet ledger
- Budget ledger
- Savings ledger
- Spend ledger
- Request ledger
- Report ledger

This allows:

- Rainy day savings %
- Budget tracking
- Category tracking
- Reports
- Future AI
- ESUSU
- Fixed savings
- Split payments

Rule:

All transactions must be recorded in ledger, not only wallet balance.

2.4 Wallet is Not Equal to Available Money

User wallet balance does not always mean user can spend all.

Because money may be:

- Saved
- Budgeted
- Locked
- Reserved
- Scheduled
- Requested
- Pending

Rule:

Available balance may be different from total balance.

Future UI may show:

- Total balance
- Available balance
- Saved balance
- Budgeted balance

2.5 Savings Must Be Intentional

Savey savings are not only manual.

Types:

- Rainy day (auto %)
- Flexible
- Fixed
- ESUSU
- BSG
- Future goal saving

Rule:

Savings must have type and rule.

Examples:

Rainy day:

% of spend goes to savings

Fixed:

Locked until date

ESUSU:

Group cycle

Flexible:

Free save

2.6 Requests Are Part of Funding Flow

Request is not just chat feature.

It is part of finance flow.

Request types:

- Request from saver
- Request from friend

Rules:

- Request must create record
- Request must have status
- Request must affect ledger after approval
- Request must be trackable

Possible statuses:

- Pending
- Approved
- Declined
- Expired
- Cancelled

2.7 Funding Must Have Source

Top up must always have source.

Sources:

- Bank transfer
- Card
- USSD (future)
- Request (future)
- Wallet transfer (future)

Rule:

All funding must be traceable to source.

Needed for:

- Reports
- Compliance
- Fraud check
- Ledger

2.8 Every Transaction Must Be Categorized

Transactions must have:

- Type
- Budget

- Category
- Source
- Destination
- Date
- Status

Used for:

- Reports
- Filters
- AI later
- Budget tracking
- Spending analysis

Already supported by:

Transaction history filter.

2.9 Reports Must Be Accurate

Reports depend on:

- Budget
- Savings
- Spend
- Income
- Category
- Date

Rule:

No transaction should bypass ledger.

Otherwise reports break.

2.10 System Must Support Future Features

Current system must allow:

- Loan
- Insurance
- Investment
- AI budgeting
- AI savings
- Group saving
- Family wallet

- Shared wallet
- Sub wallets

So architecture must stay modular.

2.11 Compliance & Audit Friendly

Because Savey is fintech.

Rules:

- All transactions logged
- All balance changes logged
- All funding tracked
- All savings tracked
- All requests tracked

Needed for:

- Anchor
- Future license
- Audit
- Investors
- Banks

3. SYSTEM MODULE ARCHITECTURE

This will list all modules:

- Onboarding
- Wallet
- Funding
- Spend
- Request
- Budget
- Savings
- Reports
- History
- Bills
- Beneficiaries
- Notifications
- Security
- Ledger
- Limits

This section is very important for dev + PM.

Continuing.

3. SYSTEM MODULE ARCHITECTURE

This section defines the **main modules inside Savey Personal**.

Each module represents a functional part of the product.

All current features and future features must belong to a module.

This architecture is important for:

- Product planning
- Dev structure
- Jira epics
- API structure
- Ledger logic
- Future expansion

Savey Personal is divided into the following modules.

3.1 Onboarding & Verification Module

Handles user creation and identity verification.

Functions:

- Signup
- Login
- OTP verification
- KYC / BVN / NIN
- Anchor verification
- PIN setup
- Security setup

Rules:

- User must be verified before full access
- Limits may apply before verification
- Wallet created after verification

Future:

- Face verification
- Tier limits
- Address verification

3.2 Wallet Module

Stores user funds.

Functions:

- Main wallet balance
- Wallet ledger
- Credit / debit
- Internal transfers
- Available balance calculation

Wallet is affected by:

- Top up
- Spend
- Savings
- Request
- Bills
- Withdrawals
- Scheduled payments

Important rule:

Wallet balance is not equal to spendable balance.

Future:

- Sub wallets
- Family wallet
- Shared wallet
- Business wallet

3.3 Funding (Top Up) Module

Handles all incoming money.

Current methods:

- Bank transfer 
- Card funding 
- USSD  (planned)

Future methods:

- Request funding
- Wallet transfer
- Goal funding
- External payment link

Rules:

- Funding must have source
- Funding must create ledger entry
- Funding must update wallet

Funding sources stored for reports.

3.4 Spend Module

Handles all outgoing money.

Spend requires:

- Budget selection
- Category
- Destination

Current features:

- Send to bank
- Send to saver
- Pay bills (in progress)
- Scheduled payment
- Split payment
- Save beneficiary

Rules:

- Budget must be selected before spend
- Spend must create ledger record
- Spend must update reports
- Spend may trigger savings %

Future:

- Spend from goal
- Spend from sub wallet
- Spend limits
- Smart spend

3.5 Request Module

Handles requesting money.

Types:

- Request from saver
- Request from friend

Flow:

- Create request
- Send request
- Receiver approves / declines
- Transfer happens after approval

Request must have:

- Status
- Sender
- Receiver
- Amount
- Note
- Date

Statuses:

- Pending
- Approved
- Declined
- Cancelled
- Expired

Future:

- Request link
- Group request
- Budget request
- Goal funding request

3.6 Budget Module

Core part of Savey.

Supports:

- Prebuilt ledger systems
- Custom budget categories
- Budget allocation
- Budget tracking

Budget used by:

- Spend

- Reports
- Savings
- AI later

Rules:

- Spend must belong to budget
- Budget must track usage
- Budget must support categories

Future:

- Monthly budget
- Weekly budget
- Goal budget
- Family budget

3.7 Savings Module

Handles all saving types.

Current:

- Rainy day savings
- Flexible savings

In progress:

- BSG
- ESUSU
- Fixed savings

Savings rules:

- Savings may lock funds
- Savings may auto deduct
- Savings may be percentage
- Savings must have ledger

Types:

Rainy day:

% of spending

Flexible:

manual save

Fixed:

locked

ESUSU:
group cycle

BSG:
structured saving group

Future:

- Goal savings
- Auto rules
- AI saving
- Shared savings

3.8 Transaction History Module

Shows all transactions.

Supports filter by:

- Income
- Expense
- Budget
- Category
- Date
- Type

Must include:

- Funding
- Spend
- Savings
- Request
- Bills
- Transfer

History must come from ledger.

Future:

- Export
- Search
- Tags

3.9 Reports Module

Shows financial summary.

Includes:

- Spending report
- Savings report
- Budget report
- Income vs expense

Reports depend on:

- Budget
- Ledger
- Savings
- Spend

Future:

- AI report
- Monthly summary
- Alerts
- Insights

3.10 Bills & Subscription Module

In progress.

Features:

- Utilities
- Bills
- Subscriptions

Rules:

- Must use spend flow
- Must select budget
- Must record category

Future:

- Auto pay
- Reminders
- Scheduled bills

3.11 Beneficiaries Module

Stores saved recipients.

Supports:

- Bank beneficiary
- Saver beneficiary

Used in:

- Spend
- Request
- Split payment

Future:

- Favorites
- Groups
- Frequent contacts

3.12 Ledger System (Internal Module)

Most important module.

Tracks:

- Wallet ledger
- Budget ledger
- Savings ledger
- Request ledger
- Report ledger

Every transaction must enter ledger.

Needed for:

- Reports
- Savings %
- Budget tracking
- AI later
- Audit

3.13 Security & Limits Module

Handles:

- PIN
- Limits
- Verification level
- Fraud checks

Future:

- Tier limits
- Daily limits
- Spend limits
- Savings lock rules

3.14 Future Modules (Not yet built)

Will be added later.

- Loan
- Insurance
- Investment
- AI budgeting
- AI savings
- AI financial coach
- Family wallet
- Group wallet
- Business wallet

System must support them.

4. FUNDING MODULE DOCUMENTATION (TOP UP)

We will fully document:

- Bank funding
- Card funding
- USSD
- Rules
- Ledger
- Flow
- Edge cases
- Future funding types

4. FUNDING MODULE (TOP UP) — FULL DOCUMENTATION

The Funding Module handles all ways money enters the user wallet.

Funding must always come from a valid source and must be recorded in the ledger.

Funding affects:

- Wallet balance
- Reports
- Budget allocation

- Savings rules
- Transaction history
- Limits
- Compliance logs

4.1 Purpose of Funding Module

The purpose of funding is to allow users to add money into Savey wallet using supported methods.

Funding must:

- Be traceable
- Be recorded
- Update wallet
- Update ledger
- Appear in history
- Appear in reports

Funding must not bypass the ledger system.

4.2 Current Supported Funding Methods

4.2.1 Bank Transfer (DONE)

User can fund wallet via bank transfer.

Flow:

1. User taps Top Up
2. User selects Bank Transfer
3. System shows account details
4. User sends money from bank
5. System detects payment
6. Wallet credited
7. Ledger updated
8. Transaction saved

Required data:

- Amount
- Bank
- Reference
- User ID
- Transaction ID

Rules:

- Must verify payment
- Must not double credit
- Must log source

4.2.2 Card Funding (DONE)

User can fund wallet using debit card.

Flow:

1. User taps Top Up
2. Select Card
3. Enter card details
4. Payment processed
5. Wallet credited
6. Ledger updated
7. Transaction saved

Rules:

- Must confirm payment success
- Must store transaction reference
- Must support retry
- Must support failure state

Future:

- Save card
- Multiple cards
- Card history

4.2.3 USSD Funding (NOT DONE)

Planned feature.

User funds wallet via USSD.

Flow:

1. User taps Top Up
2. Select USSD
3. Select bank
4. Show USSD code
5. User dials code

6. Payment confirmed
7. Wallet credited
8. Ledger updated

Rules:

- Must confirm payment
- Must handle delay
- Must handle timeout

4.3 Future Funding Methods

These are planned.

Request funding

User requests money → approval → wallet credited.

Wallet transfer

From another Savey wallet.

Goal funding

Fund directly to savings / goal.

External link

User sends link → payer funds.

4.4 Funding Ledger Rules

Every funding must create ledger entry.

Ledger fields:

- User ID
- Amount
- Source
- Method
- Date
- Status
- Reference

Ledger types:

- Wallet credit

- Funding record
- Report record

No funding should skip ledger.

4.5 Wallet Update Rules

After successful funding:

Wallet balance += amount

But:

Available balance may change depending on:

- Savings rules
- Budget allocation
- Locks
- Scheduled payments

Future:

Rainy day saving may deduct automatically.

4.6 Funding & Savings Interaction

Funding may trigger savings.

Example:

Rainy day saving active → % goes to savings.

Flow:

Funding → Wallet → Savings rule → Savings ledger → Wallet updated

Must be supported by system.

4.7 Funding & Budget Interaction

Funding does NOT automatically go to budget.

User must allocate later.

Reason:

Savey is structure-first.

Possible future:

Auto allocate.

4.8 Funding & Reports

Funding must appear as:

Income

Reports must show:

- Total income
- Income by date
- Income by source

History filter must support:

Income / credit.

4.9 Funding Edge Cases

Must handle:

- Duplicate payment
- Payment delay
- Payment failure
- Network error
- Wrong reference
- Partial payment
- Timeout

Wallet must not credit twice.

4.10 Funding Limits (Future)

Limits may apply:

- Daily limit
- Per transaction limit
- Tier limit
- KYC limit

Needed for compliance.

4.11 Compliance Requirements

Funding must log:

- Source
- Reference
- Amount
- User
- Date
- Status

Needed for:

- Anchor
- Audit
- Fraud check
- License later

4.12 Future Improvements

- Auto funding rules
- Recurring funding
- Scheduled funding
- Goal funding
- Split funding
- AI allocation

5. SPEND MODULE — FULL DOCUMENTATION

This is one of the most important parts because:

Spend requires budget
Spend supports split
Spend supports schedule
Spend supports bill pay
Spend supports saver transfer
Spend supports beneficiary

This section will be long.

Continuing.

5. SPEND MODULE — FULL DOCUMENTATION

The Spend Module handles all outgoing money from the user wallet.

This is one of the core features of Savey because spending must follow the **structure-first rule**.

Spend affects:

- Wallet balance
- Budget ledger
- Savings rules
- Reports
- Transaction history
- Beneficiaries
- Scheduled payments
- Split payments

Spend must always go through the ledger system.

5.1 Purpose of Spend Module

The purpose of the Spend module is to allow users to send or pay money while maintaining financial structure.

Spend must:

- Belong to a budget
- Have a category
- Have a destination
- Be recorded in ledger
- Update reports
- Update savings rules

Savey rule:

User must select budget before spending.

This rule is mandatory.

5.2 Spend Entry Flow

Flow:

1. User taps Spend
2. User selects budget
3. User selects spend type

4. User enters amount
5. User selects destination
6. User confirms
7. Transaction processed
8. Ledger updated
9. Reports updated
10. Savings rule applied (if active)

5.3 Spend Types (Current)

5.3.1 Send to Other Bank (DONE)

User can transfer to external bank account.

Flow:

1. Select budget
2. Select Send to Bank
3. Enter account / select beneficiary
4. Enter amount
5. Confirm
6. Transfer processed

Rules:

- Must select budget
- Must check balance
- Must save ledger
- Must save beneficiary if chosen

5.3.2 Send to Saver (DONE)

User sends to another Savey user.

Flow:

1. Select budget
2. Select Send to Saver
3. Search saver
4. Enter amount
5. Confirm
6. Transfer processed

Rules:

- Must select budget

- Must verify saver
- Must update both wallets
- Must update ledger for both users

Future:

- Frequent saver list
- Contacts sync

5.3.3 Pay Bills (IN PROGRESS)

User pays utilities, bills, subscriptions.

Examples:

- Airtime
- Data
- Electricity
- TV
- Internet
- Subscription

Flow:

1. Select budget
2. Select bill type
3. Enter details
4. Enter amount
5. Confirm
6. Payment processed

Rules:

- Must belong to budget
- Must record category
- Must appear in reports

Future:

- Auto pay
- Saved bills
- Scheduled bills

5.3.4 Scheduled Payment (DONE)

User can schedule spending.

Flow:

1. Select budget
2. Enter amount
3. Select date
4. Save schedule

System will:

- Check balance on date
- Process spend
- Update ledger

Rules:

- Must reserve funds (future)
- Must log schedule
- Must log execution

Future:

- Recurring payments
- Reminders

5.3.5 Split Payment (DONE)

User can split amount.

Example:

- Send to multiple people
- Split expense

Flow:

1. Select budget
2. Enter total
3. Add recipients
4. Confirm

Rules:

- Must track each split
- Must log separately
- Must update ledger for each

Needed for:

- group payments

- contributions
- shared expenses

5.3.6 Beneficiary Save (DONE)

User can save recipient.

Types:

- Bank beneficiary
- Saver beneficiary

Used in:

- Send to bank
- Send to saver
- Bills (future)

Rules:

- Must store securely
- Must allow delete
- Must allow edit

Future:

- Favorites
- Frequent
- Groups

5.4 Spend Ledger Rules

Every spend must create:

- Wallet debit
- Budget debit
- Report record
- History record

Ledger fields:

- User
- Amount
- Budget
- Category
- Type
- Destination

- Date
- Status

No spend should bypass ledger.

5.5 Spend & Budget Interaction

Budget must be selected.

Spend reduces budget usage.

Example:

Food budget = 50k

Spend = 5k

Remaining = 45k

Rules:

- Budget must track usage
- Budget must not go negative (optional future)
- Budget must support categories

5.6 Spend & Savings Interaction

Spend may trigger savings.

Example:

Rainy day = 5%

Spend = 10k

Savings = 500

Wallet = 9500

Flow:

Spend → Savings rule → Savings ledger

Must be supported.

5.7 Spend & Reports

Spend must appear in:

- Expense report
- Budget report
- Category report

Filters:

- Date
- Budget
- Type
- Category

History must show spend.

5.8 Spend Validation Rules

Before spend:

Check:

- Wallet balance
- Available balance
- Savings locks
- Schedule locks
- Limits
- Budget selected

Fail if:

- Not enough balance
- Budget not selected
- Invalid recipient

5.9 Spend Limits (Future)

Possible limits:

- Daily limit
- Per transfer limit
- Bill limit
- Tier limit

Needed for compliance.

5.10 Future Spend Features

Planned:

- Spend from goal
- Spend from savings
- Spend rules
- AI spend warning
- Budget warning
- Smart spend
- Family spend
- Shared spend

5.11 Compliance / Audit

Spend must log:

- User
- Amount
- Destination
- Budget
- Date
- Status
- Reference

Needed for:

- Anchor
- Audit
- Fraud check

6. REQUEST MODULE — FULL DOCUMENTATION

The Request Module allows users to request money from another person inside the Savey system.

This feature is part of the structured finance flow and is treated as a funding-related transaction.

Request is different from transfer because:

- Transfer = sender initiates
- Request = receiver initiates

Request must be tracked, approved, and recorded in the ledger.

6.1 Purpose of Request Module

The purpose of the Request module is to allow users to ask another user to send money in a structured and trackable way.

Use cases:

- Request from friend
- Request from saver
- Contribution request
- Shared expense request
- Allowance request
- Family request (future)
- Group request (future)

Request must support:

- Status
- Approval
- History
- Ledger
- Reports

6.2 Request Types (Current)

6.2.1 Request from Saver (DESIGN DONE)

User requests money from another Savey user.

Flow:

1. User taps Request
2. Select Request from Saver
3. Select saver
4. Enter amount
5. Add note (optional)
6. Send request
7. Saver receives notification
8. Saver approves or declines
9. If approved → transfer happens
10. Ledger updated
11. History updated

Rules:

- Saver must exist
- Request must have amount
- Request must have sender & receiver

- Request must have status

6.2.2 Request from Friend (DESIGN DONE)

User requests from a friend.

Friend may be:

- Savy user
- External (future)

Current flow (Savy user):

1. Select friend
2. Enter amount
3. Send request
4. Friend approves / declines
5. Transfer happens

Future flow (external):

- Request link
- Payment link
- Bank funding

6.3 Request Status

Every request must have status.

Statuses:

- Pending
- Approved
- Declined
- Cancelled
- Expired

Rules:

Pending → waiting for action

Approved → transfer done

Declined → closed

Cancelled → requester cancelled

Expired → timeout

Status must be stored in DB.

6.4 Request Ledger Rules

Request must create record even before approval.

Ledger stages:

1. Request created
2. Request pending
3. Request approved
4. Wallet transfer
5. Ledger update
6. History update
7. Report update

No request should bypass ledger.

6.5 Request & Wallet Interaction

When request is sent:

Wallet does NOT change.

When request approved:

Sender wallet ↓

Receiver wallet ↑

Ledger must record both.

6.6 Request & Budget Interaction (Future)

Future:

Request may belong to budget.

Example:

Request for food

Request for rent

Request for group savings

This will allow better reports.

Not required now but system should allow.

6.7 Request & Savings Interaction

Possible future:

Request may go to savings.

Example:

Fund my goal

Fund my savings

Fund ESUSU

Fund BSG

System should support.

6.8 Request Notifications

Receiver must get notification.

Types:

- In-app
- Push
- Email (future)

Notification must show:

- Who requested
- Amount
- Note
- Action buttons

Approve / Decline

6.9 Request Validation Rules

Before request:

Check:

- User exists
- Amount valid
- Not self request
- Not blocked user

Before approval:

Check:

- Balance enough
- Limits ok
- User active
- Request not expired

Fail if invalid.

6.10 Request Limits (Future)

Possible limits:

- Max request amount
- Daily requests
- Tier limits

Needed for fraud control.

6.11 Request History

Request must appear in history.

Types:

- Sent requests
- Received requests
- Approved
- Declined
- Pending

Filters:

- Status
- Date
- User
- Amount

6.12 Future Request Features

Planned:

- Request link

- Group request
- Budget request
- Goal funding request
- Family request
- Split request
- Recurring request

System must support.

6.13 Compliance / Audit

Request must log:

- Requester
- Receiver
- Amount
- Date
- Status
- Approval time
- Transfer reference

Needed for:

- Audit
- Fraud
- Support
- Disputes

7. BUDGET MODULE — FULL DOCUMENTATION

The Budget Module is one of the core systems of Savey.

Unlike normal fintech apps, Savey requires users to assign spending to a budget before money can be spent.

The budget system allows users to plan, allocate, and track money using a ledger-based structure.

Budget affects:

- Spend
- Reports
- Savings rules
- Categories
- Transaction history
- Future AI
- Future family/group features

7.1 Purpose of Budget Module

The purpose of the Budget module is to help users control how money is used by assigning spending to predefined or custom categories.

Budget allows users to:

- Plan spending
- Track spending
- Limit spending
- Categorize spending
- View reports
- Build discipline

Savey rule:

Every spend must belong to a budget.

7.2 Budget Types

Savey supports two types of budgets.

7.2.1 Prebuilt Budget Ledger (DONE)

System provides predefined categories.

Examples:

- Food
- Transport
- Rent
- Bills
- Shopping
- Savings
- Giving
- Misc

User can select from these.

Benefits:

- Easy setup
- Standard reports
- Better analytics

7.2.2 Custom Budget Categories (DONE)

User can create own categories.

Examples:

- Business
- Side hustle
- School fees
- Project
- Family support
- Events

Rules:

- User must name category
- Category must belong to user
- Category must appear in spend

Custom budgets must also use ledger.

7.3 Budget Allocation

User may allocate money to budget (future enhancement).

Example:

Wallet = 100k

Food = 20k

Bills = 30k

Transport = 10k

Savings = 40k

This allows:

- Better control
- Better reports
- Future AI

Current system may not fully enforce allocation, but ledger must support it.

7.4 Budget & Spend Interaction

Spend must require budget selection.

Flow:

1. User taps Spend
2. User selects budget
3. User enters amount
4. Spend processed
5. Budget ledger updated

Budget must track:

- Total spent
- Remaining (future)
- Category usage

7.5 Budget Ledger Rules

Each budget must have ledger.

Ledger fields:

- Budget ID
- User ID
- Amount spent
- Amount allocated (future)
- Category
- Date
- Transaction ID

Spend must update:

Budget ledger + Wallet ledger

Never wallet only.

7.6 Budget & Reports Interaction

Reports use budget data.

Examples:

- Spending by category
- Spending by budget
- Monthly report
- Expense breakdown

Without budget ledger → reports break.

Rule:

Budget must always update report ledger.

7.7 Budget & Savings Interaction

Budget may affect savings.

Example:

Rainy day saving uses spend amount.

Spend → Budget → Savings %

Future:

Budget-specific savings.

Example:

Save 5% of food spend.

System should support later.

7.8 Budget Validation Rules

Before spend:

Check:

- Budget selected
- Budget exists
- Category valid

Future checks:

- Budget limit
- Budget remaining
- Budget locked

Fail if invalid.

7.9 Budget Limits (Future)

Possible rules:

- Budget limit per month
- Budget limit per category

- Budget warning
- Budget lock

Examples:

Food limit = 30k

If spend > 30k → warning / block.

System should allow.

7.10 Budget History

Budget must track history.

User should see:

- Total spent
- Category breakdown
- Monthly spend
- Previous months (future)

Needed for:

Reports

AI

Planning

7.11 Future Budget Features

Planned:

- Monthly budget
- Weekly budget
- Recurring budget
- Shared budget
- Family budget
- Group budget
- Goal budget
- AI budget suggestion

System must allow.

7.12 Compliance / Audit

Budget must log:

- Budget ID
- Transaction ID
- Amount
- Category
- Date

Needed for:

- Reports
- Audit
- Debug
- Support

7.13 Importance of Budget in Savey

Budget is not optional.

Budget is the core of:

- Spend
- Reports
- Savings %
- Discipline
- AI later

Savey without budget = normal wallet.

Savey with budget = structured finance.

8. SAVINGS MODULE — FULL DOCUMENTATION

The Savings Module allows users to store money intentionally using different saving types. Savings in Savey are not just storing money — they follow rules, ledgers, and structures.

Savings may be:

- Automatic
- Manual
- Locked
- Flexible
- Group-based
- Rule-based

Savings affect:

- Wallet balance
- Available balance
- Reports
- Ledger
- Budget (future)
- Requests (future)
- AI (future)

8.1 Purpose of Savings Module

The purpose of the Savings module is to help users save money in a structured way.

Users should be able to:

- Save manually
- Save automatically
- Lock money
- Save by rule
- Save by goal
- Save with others (future)

Save rule:

Savings must have a type and rule.

8.2 Savings Types (Current & In Progress)

8.2.1 Rainy Day Savings (DONE)

Automatic saving based on spending.

User sets percentage.

Example:

Rainy day = 5%

Spend = 10,000

Savings = 500

Wallet = 9,500

Flow:

1. User activates rainy day
2. User sets %
3. Every spend triggers saving
4. Savings ledger updated

5. Wallet reduced

Rules:

- Must not exceed balance
- Must update ledger
- Must appear in history

This requires ledger support.

8.2.2 Flexible Savings (DONE)

User can save anytime.

Features:

- Save manually
- Withdraw anytime
- No lock

Flow:

1. User selects save
2. Enter amount
3. Move to savings
4. Ledger updated

Withdraw:

1. Select savings
2. Withdraw
3. Wallet credited
4. Ledger updated

Rules:

- Must track savings balance
- Must track wallet balance
- Must log transaction

8.2.3 Fixed Savings (IN PROGRESS)

Money locked until date.

Features:

- Set amount

- Set duration
- Lock funds

Rules:

- Cannot withdraw before date
- Must store lock date
- Must update ledger

Future:

- Interest
- Penalty
- Early break

8.2.4 ESUSU (IN PROGRESS)

Group saving cycle.

Features:

- Multiple users
- Contribution cycle
- Payout rotation

Rules:

- Must track members
- Must track contribution
- Must track payout order
- Must use ledger

Needed for:

Group saving
Community saving
Family saving

8.2.5 BSG (IN PROGRESS)

Structured savings group.

Features:

- Multiple members
- Rules
- Target saving

- Tracking

Rules:

- Must log each contribution
- Must track group ledger
- Must track user ledger

Future:

- Admin
- Roles
- Approval

8.3 Savings Ledger Rules

Each saving must create ledger entry.

Ledger types:

- Wallet debit
- Savings credit
- Savings withdrawal
- Auto saving
- Group saving

Fields:

- User
- Amount
- Type
- Date
- Status
- Reference

Savings must never bypass ledger.

8.4 Wallet & Savings Interaction

When saving:

Wallet ↓
Savings ↑

When withdrawing:

Savings ↓
Wallet ↑

Wallet must not show saved money as spendable.

Future UI:

- Total balance
 - Available balance
 - Saved balance
-

8.5 Savings & Spend Interaction

Spend may trigger savings.

Rainy day example:

Spend → Savings %

Flow:

Spend → Wallet → Savings → Ledger

System must support.

8.6 Savings & Reports

Savings must appear in:

- Savings report
- Total saved
- Savings history

Reports must show:

- Total saved
 - Total withdrawn
 - Total auto saved
 - Total locked
-

8.7 Savings Validation Rules

Before saving:

Check:

- Balance enough
- Not locked
- Valid amount

Before withdrawal:

Check:

- Not fixed
- Not locked
- Enough savings

Fail if invalid.

8.8 Savings Limits (Future)

Possible limits:

- Max saving
- Daily saving
- Fixed limit
- Group limit

Needed for compliance.

8.9 Future Savings Features

Planned:

- Goal savings
- Auto save rules
- AI save suggestion
- Budget save
- Family save
- Shared save
- Child save
- Business save

System must allow.

8.10 Compliance / Audit

Savings must log:

- User

- Amount
- Type
- Date
- Status

Needed for:

Audit

Support

Fraud check

Reports

8.11 Importance of Savings in Savey

Savings is one of the pillars of Savey.

Savey is built on:

- Budget
- Savings
- Structure
- Ledger
- Discipline

Without savings → Savey becomes wallet.

With savings → Savey becomes financial system.

✅ Section 8 complete.

Next section:

9. TRANSACTION HISTORY & REPORT MODULE

This will cover:

- History filters
- Income / expense
- Budget filter
- Report system
- Analytics
- Future AI reports

This connects everything together.

Continuing.

8. SAVINGS MODULE — FULL DOCUMENTATION

The Savings Module allows users to store money intentionally using different saving types. Savings in Savey are not just storing money — they follow rules, ledgers, and structures.

Savings may be:

- Automatic
- Manual
- Locked
- Flexible
- Group-based
- Rule-based

Savings affect:

- Wallet balance
- Available balance
- Reports
- Ledger
- Budget (future)
- Requests (future)
- AI (future)

8.1 Purpose of Savings Module

The purpose of the Savings module is to help users save money in a structured way.

Users should be able to:

- Save manually
- Save automatically
- Lock money
- Save by rule
- Save by goal
- Save with others (future)

Savey rule:

Savings must have a type and rule.

8.2 Savings Types (Current & In Progress)

8.2.1 Rainy Day Savings (DONE)

Automatic saving based on spending.

User sets percentage.

Example:

Rainy day = 5%

Spend = 10,000

Savings = 500

Wallet = 9,500

Flow:

1. User activates rainy day
2. User sets %
3. Every spend triggers saving
4. Savings ledger updated
5. Wallet reduced

Rules:

- Must not exceed balance
- Must update ledger
- Must appear in history

This requires ledger support.

8.2.2 Flexible Savings (DONE)

User can save anytime.

Features:

- Save manually
- Withdraw anytime
- No lock

Flow:

1. User selects save
2. Enter amount
3. Move to savings
4. Ledger updated

Withdraw:

1. Select savings
2. Withdraw

3. Wallet credited
4. Ledger updated

Rules:

- Must track savings balance
 - Must track wallet balance
 - Must log transaction
-

8.2.3 Fixed Savings (IN PROGRESS)

Money locked until date.

Features:

- Set amount
- Set duration
- Lock funds

Rules:

- Cannot withdraw before date
- Must store lock date
- Must update ledger

Future:

- Interest
 - Penalty
 - Early break
-

8.2.4 ESUSU (IN PROGRESS)

Group saving cycle.

Features:

- Multiple users
- Contribution cycle
- Payout rotation

Rules:

- Must track members
- Must track contribution
- Must track payout order
- Must use ledger

Needed for:

Group saving
Community saving
Family saving

8.2.5 BSG (IN PROGRESS)

Structured savings group.

Features:

- Multiple members
- Rules
- Target saving
- Tracking

Rules:

- Must log each contribution
- Must track group ledger
- Must track user ledger

Future:

- Admin
 - Roles
 - Approval
-

8.3 Savings Ledger Rules

Each saving must create ledger entry.

Ledger types:

- Wallet debit
- Savings credit
- Savings withdrawal
- Auto saving
- Group saving

Fields:

- User
- Amount
- Type
- Date

- Status
- Reference

Savings must never bypass ledger.

8.4 Wallet & Savings Interaction

When saving:

Wallet ↓
Savings ↑

When withdrawing:

Savings ↓
Wallet ↑

Wallet must not show saved money as spendable.

Future UI:

- Total balance
 - Available balance
 - Saved balance
-

8.5 Savings & Spend Interaction

Spend may trigger savings.

Rainy day example:

Spend → Savings %

Flow:

Spend → Wallet → Savings → Ledger

System must support.

8.6 Savings & Reports

Savings must appear in:

- Savings report
- Total saved

- Savings history

Reports must show:

- Total saved
 - Total withdrawn
 - Total auto saved
 - Total locked
-

8.7 Savings Validation Rules

Before saving:

Check:

- Balance enough
- Not locked
- Valid amount

Before withdrawal:

Check:

- Not fixed
- Not locked
- Enough savings

Fail if invalid.

8.8 Savings Limits (Future)

Possible limits:

- Max saving
- Daily saving
- Fixed limit
- Group limit

Needed for compliance.

8.9 Future Savings Features

Planned:

- Goal savings

- Auto save rules
- AI save suggestion
- Budget save
- Family save
- Shared save
- Child save
- Business save

System must allow.

8.10 Compliance / Audit

Savings must log:

- User
- Amount
- Type
- Date
- Status

Needed for:

Audit

Support

Fraud check

Reports

8.11 Importance of Savings in Savey

Savings is one of the pillars of Savey.

Savey is built on:

- Budget
- Savings
- Structure
- Ledger
- Discipline

Without savings → Savey becomes wallet.

With savings → Savey becomes financial system.

 Section 8 complete.

Next section:

9. TRANSACTION HISTORY & REPORT MODULE

This will cover:

- History filters
- Income / expense
- Budget filter
- Report system
- Analytics
- Future AI reports

This connects everything together.

Continuing.

9. TRANSACTION HISTORY & REPORT MODULE — FULL DOCUMENTATION

The History & Report Module is responsible for showing users all financial activities and summaries inside Savey.

This module depends on the ledger system and must reflect all transactions accurately.

This module includes:

- Transaction history
- Filters
- Reports
- Spending summary
- Savings summary
- Income summary
- Future analytics / AI

History and Reports must always match ledger records.

9.1 Purpose of History Module

The purpose of the history module is to allow users to view all past financial activities.

History must include:

- Funding
- Spend
- Savings
- Requests
- Bills

- Transfers
- Withdrawals
- Scheduled payments

Rule:

Every transaction must appear in history.

No transaction should bypass history.

9.2 Transaction Types

Each transaction must have type.

Examples:

- Income
- Expense
- Savings
- Withdrawal
- Request
- Transfer
- Bill payment
- Auto saving
- Scheduled payment

Type is used for:

- Filters
 - Reports
 - AI
 - Budget tracking
-

9.3 History Filters (DONE)

User can filter transactions.

Supported filters:

- Income
- Expenditure
- Budget
- Category
- Date
- Type

Future filters:

- Savings type
- Request status
- Bill type
- User
- Method

Filters must come from ledger data.

9.4 History Fields

Each transaction must store:

- Transaction ID
- User ID
- Amount
- Type
- Budget
- Category
- Source
- Destination
- Date
- Status
- Reference

This is required for:

Reports
Audit
Support
AI

9.5 Purpose of Report Module

Reports help user understand money usage.

Reports must show:

- Total income
- Total expense
- Total saved
- Budget usage
- Category usage

Reports must use ledger data.

9.6 Current Reports (DONE)

User can see:

- Spending report
- Savings report
- Summary

Report examples:

Total spent this month

Total saved

Budget breakdown

Income vs expense

Reports must update after every transaction.

9.7 Report Categories

Reports may be grouped by:

- Budget
- Category
- Date
- Type
- Savings
- Income

Example:

Food → 20k

Bills → 30k

Transport → 10k

This comes from budget ledger.

9.8 Report Calculation Rules

Reports must use ledger only.

Never calculate from wallet balance.

Correct flow:

Transaction → Ledger → Report → History

Not:

Wallet → Report

This prevents errors.

9.9 History & Budget Interaction

History must show budget.

Example:

Spend

Amount = 5k

Budget = Food

Needed for:

Reports

Filters

Analytics

9.10 History & Savings Interaction

Savings must appear in history.

Examples:

Rainy day saving

Flexible saving

Fixed saving

Withdrawal

All must show.

9.11 History & Request Interaction

Request must appear.

Examples:

Request sent

Request received

Request approved

Request declined

Must store status.

9.12 History & Funding Interaction

Funding must appear as income.

Examples:

Bank funding
Card funding
USSD funding
Request funding

Reports must count as income.

9.13 Report Accuracy Rules

Reports must always match:

Wallet + Savings + Budget + Ledger

If ledger wrong → report wrong.

Rule:

Ledger is source of truth.

9.14 Future Reports

Planned:

- Monthly report
- Weekly report
- AI summary
- Spending trend
- Saving trend
- Budget alert
- Smart insight

Example:

"You spent more on food this month"

System must support.

9.15 Export / Share (Future)

User may export:

- PDF
- CSV
- Statement

Needed for:

Business
Audit
Loans
Proof of income

9.16 Compliance / Audit

History must store:

- All transactions
- All references
- All status
- All dates

Needed for:

Support
Fraud check
Audit
License later

9.17 Importance of History & Reports

This module connects all modules:

Funding
Spend
Budget
Savings
Request
Bills

Without history → no report
Without report → no structure
Without structure → no Savey

✓ Section 9 complete.

Next section:

10. BILLS, BENEFICIARIES & SCHEDULED PAYMENTS MODULE

This will cover:

- Bills
- Subscriptions
- Beneficiaries
- Scheduled spend
- Split payments

This is part of Spend but needs its own spec.

Continuing.

10. BILLS, BENEFICIARIES & SCHEDULED PAYMENTS MODULE — FULL DOCUMENTATION

This module handles extended spending features that are built on top of the Spend module. These include:

- Bill payments
- Subscriptions
- Saved beneficiaries
- Scheduled payments
- Split payments

These features must still follow the core Savey rules:

- Budget must be selected
- Ledger must update
- Reports must update
- Savings rules must apply
- Wallet must update correctly

This module is part of Spend but has additional logic.

10.1 Bills & Subscription Payments

This feature allows users to pay for services directly inside Savey.

Examples:

- Airtime
- Data
- Electricity
- TV subscription
- Internet
- Utility bills
- Future subscriptions

Status: Design done / In progress

10.1.1 Bill Payment Flow

1. User taps Spend
2. User selects Budget
3. User selects Pay Bills
4. User selects bill type
5. User enters details
6. User enters amount
7. User confirms
8. Payment processed
9. Ledger updated
10. Report updated

Rules:

- Budget must be selected
 - Category must be stored
 - Must record provider
 - Must record reference
-

10.1.2 Bill Ledger Rules

Each bill must create:

- Wallet debit
- Budget debit
- Report record
- History record

Fields:

- Bill type

- Provider
 - Amount
 - Budget
 - Date
 - Status
 - Reference
-

10.1.3 Future Bill Features

- Auto pay
- Reminder
- Recurring bills
- Saved bills
- Subscription tracking

System must support.

10.2 Beneficiaries Module

Allows user to save recipients.

Types:

- Bank beneficiary
- Saver beneficiary

Used in:

- Send to bank
- Send to saver
- Split payment
- Bills (future)

Status: Done

10.2.1 Beneficiary Fields

- Name
- Account number
- Bank
- Saver ID
- User ID
- Date created

Must be secure.

10.2.2 Beneficiary Rules

- Must belong to user
- Must be editable
- Must be deletable
- Must be selectable

Future:

- Favorite
 - Frequent
 - Groups
-

10.3 Scheduled Payments

Allows user to send money later.

Status: Done

Examples:

- Rent later
 - Subscription later
 - Reminder payment
 - Planned spend
-

10.3.1 Scheduled Payment Flow

1. User taps Spend
2. Select budget
3. Enter amount
4. Select date
5. Save schedule

On date:

1. System checks balance
 2. System processes spend
 3. Ledger updated
 4. Report updated
-

10.3.2 Scheduled Rules

Must store:

- Date
- Amount
- Budget
- Destination
- Status

Statuses:

- Pending
 - Executed
 - Failed
 - Cancelled
-

10.3.3 Future Scheduled Features

- Recurring payments
- Monthly payment
- Weekly payment
- Reminder
- Auto pay

Needed for bills & subscriptions.

10.4 Split Payments

Allows user to split amount.

Status: Done

Examples:

- Pay multiple people
 - Share expense
 - Contribution
-

10.4.1 Split Flow

1. Select budget
2. Enter total
3. Add recipients
4. Confirm

5. Send

Ledger must record each split.

10.4.2 Split Rules

- Total must match
- Each transfer logged
- Each user ledger updated

Needed for:

Group

Friends

Family

Future ESUSU

10.5 Interaction With Budget

All these must require budget:

- Bills
- Transfer
- Split
- Schedule

Rule:

No spend without budget.

10.6 Interaction With Savings

Spend may trigger:

- Rainy day saving
- Future rules

Flow:

Spend → Savings → Ledger

Must support.

10.7 Interaction With Reports

Bills / Split / Schedule must appear in:

- Expense report
- Budget report
- Category report

Must not bypass ledger.

10.8 Compliance / Audit

Must log:

- User
- Amount
- Destination
- Budget
- Reference
- Status
- Date

Needed for:

Support
Fraud
Audit

10.9 Importance of This Module

This module makes Savey usable daily.

Without this:

Only wallet.

With this:

Real finance system.

 Section 10 complete.

Next section:

11. LEDGER SYSTEM (CORE OF SAVEY)

This is the most technical but most important part.

We will define:

- Wallet ledger
- Budget ledger
- Savings ledger
- Request ledger
- Report ledger
- Rules

This is what makes Savey different from normal fintech.

Continuing.

11. LEDGER SYSTEM — CORE OF SAVEY

The Ledger System is the foundation of Savey.

All financial operations must go through the ledger.

Savey is not a simple balance-based wallet.

Savey is a **ledger-based financial system**.

Every module depends on the ledger:

- Wallet
- Budget
- Spend
- Savings
- Request
- Reports
- Bills
- Scheduled payments
- Future loans / investment / insurance

Rule:

Ledger is the source of truth.

Wallet balance must come from ledger, not the other way.

11.1 Purpose of Ledger System

The ledger system exists to:

- Track all transactions
- Maintain accuracy

- Support reports
- Support savings rules
- Support budget tracking
- Support audit
- Support future features

Without ledger:

- Reports break
 - Budget breaks
 - Savings breaks
 - Requests break
 - AI impossible
 - Audit impossible
-

11.2 Types of Ledgers in Savey

Savey uses multiple ledgers.

11.2.1 Wallet Ledger

Tracks all money entering and leaving wallet.

Examples:

- Funding
- Spend
- Savings
- Request transfer
- Bills
- Withdrawals

Fields:

- User
- Amount
- Type
- Date
- Reference
- Status

Wallet balance = sum(wallet ledger)

11.2.2 Budget Ledger

Tracks spending per budget.

Used for:

- Reports
- Budget tracking
- Category tracking

Fields:

- Budget ID
- User ID
- Amount
- Category
- Date
- Transaction ID

Spend must update budget ledger.

11.2.3 Savings Ledger

Tracks savings.

Used for:

- Flexible savings
- Rainy day
- Fixed
- ESUSU
- BSG

Fields:

- User
- Saving type
- Amount
- Date
- Status
- Reference

Saving must update savings ledger.

11.2.4 Request Ledger

Tracks requests.

Used for:

- Pending requests

- Approved requests
- Declined requests

Fields:

- Request ID
- Sender
- Receiver
- Amount
- Status
- Date

When approved → wallet ledger updated.

11.2.5 Report Ledger

Used for reports.

May be calculated or stored.

Tracks:

- Income
- Expense
- Savings
- Budget usage

Report should come from ledger data.

11.2.6 Schedule Ledger

Tracks future transactions.

Used for:

- Scheduled payment
- Recurring payment (future)

Fields:

- Date
- Amount
- Status
- Budget
- Destination

When executed → wallet ledger updated.

11.2.7 Group / ESUSU Ledger (Future)

Used for:

- Group saving
- Contribution
- Rotation

Fields:

- Group ID
- User
- Amount
- Cycle
- Status

Must be separate ledger.

11.3 Ledger Rules

All transactions must:

1. Create ledger entry
2. Update wallet ledger
3. Update related ledger
4. Update history
5. Update reports

Never:

Wallet → Report

Always:

Ledger → Wallet → Report

11.4 Example Ledger Flow

Example: Spend

Spend → Wallet ledger (debit)
Spend → Budget ledger
Spend → Savings ledger (if rainy day)
Spend → Report ledger
Spend → History

All must happen.

11.5 Available Balance Calculation

Wallet balance is not always spendable.

Available =

Wallet

- Locked savings
- Scheduled
- Pending
- Reserved

System must support.

11.6 Ledger & Savings Interaction

Savings must not stay in wallet ledger.

Must move to savings ledger.

Example:

Wallet - 1000
Savings + 1000

Required for:

Reports

Locking

Future interest

11.7 Ledger & Budget Interaction

Spend must update:

Wallet ledger

Budget ledger

Report ledger

Budget must not depend on wallet only.

11.8 Ledger & Request Interaction

Request approved →

Sender wallet ledger –
Receiver wallet ledger +
Request ledger updated

All must log.

11.9 Ledger & Reports Interaction

Reports must read from ledger.

Not from UI.

Rule:

Ledger = truth
Report = calculation

11.10 Ledger & Future Features

Ledger must support:

- Loan
- Insurance
- Investment
- AI
- Family wallet
- Shared wallet
- Sub wallet
- Business wallet

If ledger weak → product cannot grow.

11.11 Audit & Compliance

Ledger must store:

- All transactions
- All references
- All status
- All dates
- All users

Needed for:

- Anchor
 - License
 - Audit
 - Investors
 - Banks
-

11.12 Why Savey Needs Ledger

Savey has:

- Budget
- Savings
- Requests
- Reports
- Auto saving
- Split
- Schedule

Without ledger → impossible.

Ledger makes Savey different from:

OPay

Kuda

PalmPay

Savey behaves more like:

Financial system

Accounting system

Personal finance OS

✅ Section 11 complete.

Next section:

12. FUTURE MODULES (Loan, Insurance, Investment, AI)

This will define features not built yet but part of product vision.

Perfect. This is very important and different from normal fintech loans.

Savey loan is not a random credit feature — it is a **Goal-backed, Savings-based, Behavior-based loan**.

We must document this carefully because this is part of Savey's philosophy of **structure before spending**.

We will write this inside Section 12 as the official loan model.

12. FUTURE MODULES — LOAN, INSURANCE, INVESTMENT, AI

This section defines features that are not yet built but are part of the Savey product design. These features must follow Savey core principles and must integrate with the ledger, budget, savings, and reports system.

Future modules include:

- Loan
 - Insurance
 - Investment
 - AI budgeting
 - AI savings
 - AI financial assistant
-

12.1 LOAN MODULE — GOAL-BASED LOAN SYSTEM

Savey loan is not a normal loan system.
Users cannot take loans freely.

Loans are only given to users who have shown saving discipline and have an active financial goal.

Purpose of Savey loan:

Help users complete their financial goals when unexpected situations prevent them from reaching the target.

This makes the loan system:

- Behavior-based
 - Savings-based
 - Goal-based
 - Low risk
 - Structured
-

12.1.1 Purpose of Loan Feature

The loan feature exists to help users complete planned goals, not to encourage borrowing.

Examples:

- School fees
- Rent
- Project payment
- Business target
- Medical need
- Travel plan

Loan is allowed only if user already tried to save.

12.1.2 Loan Eligibility Rules

User must meet conditions before loan is allowed.

Minimum conditions:

1. User must be an active saver
2. User must have saved for at least 3–6 months
3. User must have an active savings goal
4. User must have contributed to the goal consistently
5. User must not have unpaid loan

Optional future rules:

- Minimum saving amount
- Minimum activity
- KYC level
- Risk score

Rule:

No saving history → No loan

12.1.3 Loan Type — Goal Completion Loan

Loan is tied to a savings goal.

Example:

Goal = 500,000

Saved = 400,000

Shortfall = 100,000

Savey may give loan = 100,000

Loan is not random.

Loan = gap to complete goal.

12.1.4 Example Flow

User goal:

Pay school fees = 500k

Duration = 5 months

User saved:

Month 1 → 100k

Month 2 → 80k

Month 3 → 90k

Month 4 → 70k

Month 5 → 60k

Total = 400k

Shortfall = 100k

User can request goal loan.

System checks:

- Saving history
- Goal active
- Duration completed
- User behavior

If approved:

Loan = 100k

User completes goal.

12.1.5 Loan Flow

1. User opens Goal
2. System shows shortfall
3. System shows Loan available (if eligible)
4. User applies
5. System checks rules
6. Loan approved / rejected
7. Loan credited
8. Goal completed
9. Repayment plan created

12.1.6 Loan Ledger Rules

Loan must create ledger entries.

Entries:

Loan credit → wallet

Loan record → loan ledger

Repayment → wallet debit

Interest → loan ledger

Ledger types:

- Loan ledger
- Wallet ledger
- Report ledger

Loan must not bypass ledger.

12.1.7 Loan Repayment Rules

Repayment may be:

- Manual
- Scheduled
- Auto deduction (future)

Fields:

- Loan amount
- Interest
- Duration
- Due date
- Remaining

Must appear in reports.

12.1.8 Loan Risk Control

Loan allowed only if:

- Saving history exists
- Goal exists
- User active
- No default

- Limits ok

Future:

- Risk score
- AI scoring
- Behavior scoring

This keeps risk low.

12.1.9 Loan & Savings Interaction

Loan must be tied to:

- Goal
- Savings
- History

Loan should not exist without savings.

Rule:

Loan supports saving, not replace saving.

12.1.10 Loan & Reports

Loan must appear in:

- History
- Report
- Loan summary

User must see:

Total loan
Remaining
Interest

12.1.11 Future Loan Types (Possible)

- Goal completion loan
- Emergency loan (future)
- Salary advance (future)
- Group loan (future)
- Business loan (future)

But all must follow Savey philosophy.

12.1.12 Why This Loan Model Fits Savey

Normal fintech loan:

Borrow → Spend → Debt

Savey loan:

Save → Plan → Shortfall → Loan → Complete goal

This keeps:

- Discipline
- Low risk
- Trust
- Structure

Very aligned with Savey.

 Loan model documented.

Next section will be:

12.2 INSURANCE MODULE
12.3 INVESTMENT MODULE
12.4 AI FINANCIAL SYSTEM

These must also follow Savey structure.

Continuing.

12.2 INSURANCE MODULE — FUTURE FEATURE

The Insurance Module will allow users to protect their finances against unexpected events. Insurance in Savey will not be random product selling, but will be connected to the user's financial plans, savings, and goals.

Insurance must follow Savey philosophy:

Protection should support financial structure, not break it.

Insurance will integrate with:

- Savings
 - Goals
 - Budget
 - Loan
 - Reports
 - Ledger
-

12.2.1 Purpose of Insurance Feature

The purpose of insurance in Savey is to help users stay on track financially even when unexpected events happen.

Examples:

- Health emergency
- School fees issue
- Business loss
- Job loss
- Travel issue
- Device loss

Insurance should help user continue their plan.

12.2.2 Insurance Types (Planned)

Possible insurance products:

- Health insurance
- Life insurance
- Device insurance
- School fees insurance
- Income protection
- Goal protection
- Savings protection

These may be offered through partners.

Savey may act as:

- Distributor
 - Aggregator
 - Partner platform
-

12.2.3 Insurance & Goal Interaction

Insurance may be linked to goal.

Example:

Goal = School fees

User can add insurance to goal.

If something happens → insurance helps complete goal.

Flow:

Goal → Insurance → Event → Payout → Goal completed

This fits Savey structure.

12.2.4 Insurance & Savings Interaction

Insurance may protect savings.

Example:

Fixed saving locked → emergency → insurance pays instead.

This prevents breaking savings.

System must support.

12.2.5 Insurance & Loan Interaction

Insurance may reduce loan risk.

Example:

Goal not completed → insurance covers part → loan smaller.

Future integration possible.

12.2.6 Insurance Ledger Rules

Insurance must have ledger.

Entries:

- Premium payment
- Policy record

- Payout
- Claim status

Must appear in:

History
Reports
Ledger

12.2.7 Insurance Payment Flow

1. User selects insurance
2. Select plan
3. Pay premium
4. Policy created
5. Ledger updated

Must use spend flow.

Budget must be selected.

12.2.8 Insurance Claims (Future)

Flow:

1. Event happens
2. User submits claim
3. Partner verifies
4. Payout sent
5. Wallet credited / goal credited

Must log in ledger.

12.2.9 Insurance & Reports

Reports must show:

- Insurance paid
- Insurance received
- Premium total

Needed for full finance tracking.

12.2.10 Why Insurance Fits Savey

Savey is about:

Plan → Save → Spend → Protect

Insurance adds protection.

Without insurance:

One event breaks plan.

With insurance:

Plan continues.

12.3 INVESTMENT MODULE — FUTURE FEATURE

Investment will allow users to grow money after saving.

Investment must follow Savey rule:

Save first → then invest.

Not:

Borrow → invest.

12.3.1 Purpose of Investment Feature

Allow users to grow money safely.

Examples:

- Fixed returns
- Funds
- Treasury
- Partner investments
- Cooperative investments

Investment should come after savings.

12.3.2 Investment Eligibility

User should:

- Have savings
- Have history
- Have verified account

Future rules:

- Minimum balance
- Minimum saving months

Keeps risk low.

12.3.3 Investment Flow

1. User opens Investment
2. Select product
3. Enter amount
4. Confirm
5. Money moved from wallet / savings
6. Investment ledger updated

Must not bypass ledger.

12.3.4 Investment Ledger

Entries:

- Investment debit
- Investment record
- Return credit
- Withdrawal

Needed for:

Reports

Audit

Compliance

12.3.5 Investment & Savings Interaction

Investment may come from:

- Flexible savings
- Wallet
- Goal (future)

Must update ledger.

12.3.6 Investment & Reports

Reports must show:

- Invested amount
- Returns
- Profit
- Loss (future)

Needed for financial clarity.

12.3.7 Future Investment Features

- Auto invest
- Goal invest
- AI invest
- Group invest
- Family invest

System must support.

12.4 AI FINANCIAL SYSTEM — FUTURE FEATURE

AI will help users manage money better.

AI will use:

- Ledger data
- Budget data
- Savings data
- Spend data
- Reports

AI must not control money without rules.

AI supports structure.

12.4.1 Purpose of AI

Help users:

- Save more
- Spend better
- Avoid waste
- Reach goals
- Plan ahead

AI = assistant, not decision maker.

12.4.2 AI Features (Planned)

- Budget suggestion
- Savings suggestion
- Spend warning
- Goal prediction
- Loan eligibility
- Risk score
- Monthly summary

Example:

"You spent more on food this month"

12.4.3 AI & Ledger

AI must read ledger.

Ledger → AI → Suggestion

Never:

AI → change ledger

This keeps system safe.

12.4.4 AI & Budget

AI may suggest:

- Budget amount
- Category change
- Limit warning

Future.

12.4.5 AI & Savings

AI may suggest:

- Increase saving %
- Create goal
- Lock savings

Future.

12.4.6 AI & Loan

AI may decide eligibility.

Based on:

- Saving history
- Budget behavior
- Goal progress

Fits Savey loan model.

12.4.7 AI & Reports

AI may generate:

- Monthly report
- Advice
- Alerts

Future.

12.4.8 Why AI Fits Savey

Savey already has:

- Budget
- Savings
- Ledger
- Reports

This is perfect for AI later.

Most apps don't have enough data.

Savey does.

✓ Section 12 complete.

Next section (final major one):

13. BUSINESS MODEL & REVENUE MODEL

This will define how Savey makes money:

- Fees
- Bills
- Savings float
- Loan interest
- Insurance commission
- Investment commission

This is needed for investors.

Continuing.

13. BUSINESS MODEL & REVENUE MODEL

This section explains how Savey makes money and how the product can sustain itself long-term.

Savey is designed as a **financial structure platform**, so revenue will come from multiple financial services, not only transfers.

Revenue must follow Savey principles:

- No exploitative lending
- No hidden fees
- No forced charges
- Earn from value, not confusion

Savey revenue will come from:

- Transaction fees
 - Bills & utilities
 - Savings float
 - Loan interest
 - Insurance commission
 - Investment commission
 - Future subscriptions
-

13.1 Core Revenue Sources

13.1.1 Transaction Fees

Fees may apply to:

- Bank transfer
- Card funding
- Withdrawal
- External payment
- Bills

Rules:

- Fees must be transparent
- Fees must be logged
- Fees must appear in history

Ledger must record:

Wallet debit

Fee record

Report record

Future:

- Tiered fees
- Free limits
- Premium plan

13.1.2 Bills & Utilities Commission

Savey can earn commission from bill payments.

Examples:

- Airtime
- Data
- Electricity
- TV
- Internet
- Subscriptions

Each bill provider may pay commission.

Flow:

User pays bill → Provider → Commission → Savey

Must record:

- Bill payment
- Commission
- Provider

This can be major revenue source.

13.1.3 Savings Float Income

When users save money, funds may stay in partner accounts.

Savey may earn from:

- Interest
- Float
- Partner yield

Examples:

- Flexible savings
- Fixed savings
- ESUSU
- BSG

Rules:

- User savings must be safe
- Must comply with partner rules
- Must log savings balance

Future:

Share interest with users.

13.1.4 Loan Interest

Savey loan is goal-based, not random.

Revenue from:

- Interest
- Late fees (future)
- Processing fee (optional)

Example:

Loan = 100k
Repay = 110k

Interest must be clear.

Loan must follow rules:

- Only savers
- Only goal users
- Only eligible users

This keeps default low.

13.1.5 Insurance Commission

Savey may partner with insurance providers.

Savey earns from:

- Policy commission
- Referral fee

User flow:

User buys insurance → Provider → Commission → Savey

Savey does not need to be insurer.

This reduces risk.

13.1.6 Investment Commission

Savey may offer investment through partners.

Possible earnings:

- Commission
- Management fee
- Referral fee

Examples:

- Treasury
- Funds
- Cooperative
- Partner investments

Investment must use ledger.

13.1.7 Subscription / Premium (Future)

Possible premium plan.

Examples:

- Advanced reports
- AI insights
- Higher limits
- No fees
- Family wallet
- Business tools

Monthly fee possible.

Not required now.

13.2 Why Savey Can Make Money

Savey has multiple modules:

- Budget
- Savings
- Spend
- Bills
- Loan
- Insurance
- Investment

Most fintech only has transfer.

Savey has more revenue points.

This makes Savey sustainable.

13.3 Low Risk Model

Savey reduces risk because:

Loan only to savers

Savings tracked

Budget tracked

Ledger tracked
Reports tracked

This allows:

Better scoring
Lower default
Better trust

13.4 Long Term Revenue Strategy

Short term:

- Bills
- Transfer fees

Mid term:

- Savings float
- Loan interest

Long term:

- Insurance
- Investment
- AI
- Premium

Very strong model.

13.5 Future Business Opportunities

Possible future:

- Family finance
- Group finance
- Cooperative finance
- SME finance
- School finance
- Church finance

All possible with current architecture.

13.6 Why Savey is Different

Most apps:

Send money → earn fee

Savey:

Plan → Save → Spend → Protect → Invest → Borrow

More lifetime value per user.

13.7 Business Goal

Goal of Savey:

Not just wallet.

Goal:

Become the financial structure platform for Africa.

✅ Section 13 complete.

Next section (last major section):

14. PRODUCT ROADMAP & FUTURE EXPANSION

This will show:

- MVP
- Current
- Next
- Future
- Long term vision

Very important for investors & strategy.

Continuing.

14. PRODUCT ROADMAP & FUTURE EXPANSION

This section defines the evolution of Savey from MVP to full financial platform.
It shows what has been built, what is in progress, and what is planned.

This roadmap is important for:

- Product planning

- Investor discussions
- Team alignment
- Feature prioritization
- Long-term vision

Savey is being built in phases, not all at once.

14.1 Phase 1 — MVP (Completed)

Goal:

Build the core structure system.

Features included:

- Onboarding & verification
- Wallet
- Top up (bank, card)
- Spend with budget selection
- Send to bank
- Send to saver
- Budget system
- Custom categories
- Transaction history
- Reports
- Flexible savings
- Rainy day savings

Purpose of MVP:

- Validate structure-first finance
- Ensure ledger works
- Ensure budget works
- Ensure savings works

This phase proves the concept.

14.2 Phase 2 — Current Phase (In Progress)

Goal:

Make Savey usable daily.

Features added / being added:

- Bills / utilities
- Subscription payments
- Scheduled payments

- Split payments
- Beneficiaries
- Request from saver
- Request from friend
- Fixed savings
- ESUSU
- BSG
- Better reports
- Better filters

Purpose:

- Increase daily usage
- Increase retention
- Increase transaction volume
- Prepare for loan & insurance

This phase makes Savey a real finance app.

14.3 Phase 3 — Structure Expansion (Next)

Goal:

Strengthen financial system.

Planned features:

- Goal-based savings UI
- Budget limits
- Budget allocation
- Sub wallets
- Available balance logic
- Savings lock rules
- Request funding flow
- Goal funding
- Notifications improvements
- Limits & tiers

Purpose:

- Improve discipline
- Improve tracking
- Prepare for AI
- Prepare for loan

This phase strengthens the system.

14.4 Phase 4 — Credit & Protection

Goal:

Add financial support features.

Planned:

- Goal-based loan
- Loan ledger
- Loan repayment
- Insurance
- Insurance partner integration
- Goal protection
- Risk scoring

Important rule:

Loan only for savers

Loan only for goals

Loan only after history

Purpose:

- Help users complete goals
- Reduce risk
- Increase revenue

This phase makes Savey powerful.

14.5 Phase 5 — Investment & Growth

Goal:

Help users grow money.

Planned:

- Investment products
- Fixed return
- Partner funds
- Cooperative investment
- Auto invest
- Goal invest

Rules:

Save → then invest

Not borrow → invest

Purpose:

- Increase retention
 - Increase float
 - Increase revenue
-

14.6 Phase 6 — AI Financial System

Goal:

Use data to guide users.

Planned AI:

- Budget suggestion
- Savings suggestion
- Spending warning
- Goal prediction
- Loan eligibility
- Monthly summary
- Financial advice

AI will use:

- Ledger
- Budget
- Savings
- Reports

Savey already has the data needed.

This phase makes Savey smart.

14.7 Phase 7 — Family / Group / Shared Finance

Goal:

Support multiple users.

Planned:

- Family wallet
- Shared wallet
- Group saving
- Contribution
- Cooperative
- ESUSU advanced
- Group loan

This is big for Africa.

Architecture already supports it.

14.8 Phase 8 — Savey Business

Future product.

Features:

- Business wallet
- Business budget
- Expense tracking
- Team wallet
- Payroll
- Invoice
- Loan
- Reports

Savey Personal → Savey Business

Same ledger system.

14.9 Phase 9 — Savey Africa Vision

Long term vision.

Savey becomes:

- Financial structure platform
- Personal finance OS
- Group finance system
- SME finance system
- Credit scoring system

Possible future:

- Bank license
- Microfinance
- Cooperative network
- API platform

Goal:

Savey becomes the system people use to manage money, not just store money.

14.10 Roadmap Summary

Phase Focus

Phase MVP

1

Phase Daily usage

2

Phase Structure

3 upgrade

Phase Loan & insurance

4

Phase Investment

5

Phase AI

6

Phase Family / Group

7

Phase Business

8

Phase Savey Africa

9

14.11 Why This Roadmap Works

Because Savey started with:

Budget

Savings

Ledger

Reports

Most fintech start with transfer.

Savey started with structure.

So expansion is easier.

✓ Section 14 complete.

DOCUMENT STATUS

We now have:

1. Product overview
2. Principles
3. Architecture
4. Funding
5. Spend
6. Request
7. Budget
8. Savings
9. History & Reports
10. Bills / Schedule / Beneficiaries
11. Ledger
12. Loan / Insurance / Investment / AI
13. Business model
14. Roadmap

This is already a **real fintech product document**.

Next step (recommended)

Next we should create:

15. USER FLOW (END-TO-END FLOW OF SAVEY)

From signup → funding → budget → spend → save → request → report → loan

This will help dev, PM, and design align.

Continuing.

15. USER FLOW — END-TO-END FLOW OF SAVEY PERSONAL

This section describes the complete user journey inside Savey, from signup to advanced features.

It shows how all modules connect together.

This flow is important for:

- Product design
- Backend logic
- QA testing
- PRD writing
- UX flow
- Investor explanation

Savey flow is different from normal fintech because it enforces structure.

15.1 First Time User Flow

Step 1 — Signup

User installs app → creates account

Steps:

1. Enter phone / email
2. Verify OTP
3. Enter details
4. KYC / BVN / NIN
5. Set PIN
6. Account created
7. Wallet created

Result:

User now has wallet.

15.2 First Wallet Funding Flow

User must fund wallet before using features.

Flow:

1. Tap Top Up
2. Select method
 - Bank transfer
 - Card
 - USSD (future)

3. Complete payment
4. Wallet credited
5. Ledger updated
6. Transaction saved

User now has balance.

15.3 Budget Setup Flow

User should create budget before spending.

Flow:

1. Open Budget
2. Select prebuilt ledger OR
3. Create custom category
4. Save budget

Examples:

Food
Transport
Bills
Business
School fees

Budget is now available for spend.

15.4 Spend Flow

Spend requires budget.

Flow:

1. Tap Spend
2. Select budget
3. Select spend type
 - Bank transfer
 - Send to saver
 - Bills
 - Split
 - Scheduled
4. Enter amount
5. Confirm
6. Wallet debited
7. Budget ledger updated

8. Savings rule applied (if active)
9. Report updated
10. History updated

Spend completed.

15.5 Savings Flow

User may save money anytime.

Flexible Saving

1. Open Savings
2. Enter amount
3. Confirm
4. Wallet ↓
5. Savings ↑
6. Ledger updated

Rainy Day Saving

1. Activate rainy day
2. Set %
3. Every spend → auto save

Fixed / ESUSU / BSG

1. Create saving
2. Set rules
3. Save
4. Funds locked / tracked

Savings recorded in ledger.

15.6 Request Flow

User may request money.

Flow:

1. Tap Request
2. Select type
 - From saver
 - From friend
3. Enter amount
4. Send request

5. Receiver notified
6. Receiver approves / declines
7. If approved → transfer
8. Ledger updated
9. History updated

Request complete.

15.7 Bills Flow

User pays bills.

Flow:

1. Tap Spend
 2. Select budget
 3. Select Bills
 4. Select provider
 5. Enter details
 6. Confirm
 7. Payment processed
 8. Ledger updated
 9. Report updated
-

15.8 Scheduled Payment Flow

1. Tap Spend
2. Select budget
3. Enter amount
4. Select future date
5. Save schedule

On date:

- Check balance
 - Execute spend
 - Update ledger
-

15.9 Split Payment Flow

1. Tap Spend
2. Select budget
3. Enter total
4. Add recipients

5. Confirm
6. Each transfer logged

Ledger updated per split.

15.10 Reports Flow

User views report.

Flow:

1. Open Reports
2. Select period
3. View summary

Shows:

- Income
- Expense
- Savings
- Budget usage

Reports use ledger.

15.11 Loan Flow (Future)

Goal-based loan.

Flow:

1. User has goal
2. User saves for months
3. Goal incomplete
4. System shows loan option
5. User applies
6. System checks eligibility
7. Loan approved
8. Wallet credited
9. Goal completed
10. Repayment tracked

Loan linked to savings.

15.12 Insurance Flow (Future)

1. User opens insurance
2. Select plan
3. Pay premium
4. Policy created
5. Ledger updated

If event:

Claim → payout → wallet / goal

15.13 Investment Flow (Future)

1. User opens investment
2. Select product
3. Enter amount
4. Confirm
5. Wallet / savings debited
6. Investment ledger updated

Returns later.

15.14 AI Flow (Future)

AI reads:

- Budget
- Savings
- Spend
- Reports

AI suggests:

- Save more
- Spend less
- Adjust budget
- Loan eligible

AI does not change money automatically.

15.15 Full Savey Flow Summary

Signup → Fund → Budget → Spend → Save → Request → Report → Goal → Loan →
Protect → Invest

This is the Savey cycle.

Not just:

Signup → Transfer

This is why Savey is different.

15.16 Why This Flow Works

Because Savey enforces:

Structure
Discipline
Tracking
Planning
Saving
Protection
Growth

This is closer to real finance than normal wallets.

 Section 15 complete.

We now have a FULL product document.

Next optional section (very useful):

16. ROLES, PERMISSIONS & USER TYPES

This will define:

- User
- Verified user
- Saver
- Friend
- Group member
- Admin
- Support

Needed for backend.

APPROVAL & ACCOUNTABILITY WORKFLOW

Feature Name

Approval & Accountability Workflow (Saver Control System)

1. Summary

The Approval & Accountability Workflow allows users to invite another Savey user (“Saver”) to either monitor or control their spending activities.

This feature introduces two modes:

1. **View Mode (Accountability)** — Invited saver receives notifications about user’s spending activities.
2. **Approval Mode (Control)** — Invited saver must approve a transaction before it is executed.

This feature reinforces Savey’s core philosophy:

Structure and discipline before spending.

2. Problem Statement

Many users struggle with:

- Impulse spending
- Lack of financial discipline
- No accountability
- Overspending beyond budget

Existing fintech apps do not provide:

- Real-time accountability
 - Approval-based spending control
 - Shared financial discipline tools
-

3. Goal / Objective

- Introduce accountability into spending behavior
- Enable controlled spending for disciplined users
- Support couples, families, and partners
- Reduce unnecessary spending
- Increase user retention and engagement

Success Metrics:

- % of users enabling saver feature
 - % of transactions requiring approval
 - Reduction in impulsive spending
 - Approval response rate
-

4. Scope

In Scope

- Invite saver via email
 - Assign saver role:
 - Viewer (notifications only)
 - Approver (must approve transactions)
 - Notification system for spending
 - Approval workflow before transaction execution
 - Transaction status updates (pending approval, approved, rejected)
-

Out of Scope (Initial)

- Multiple approvers
 - Approval chains
 - Group approvals
 - External (non-Savey user approval)
 - Auto-approval rules
-

5. User Stories

Viewer Mode

As a user,
I want someone to see my spending activities,
So that I can stay accountable.

Approval Mode

As a user,
I want someone to approve my spending,
So that I avoid unnecessary or impulsive expenses.

Approver

As a saver,
I want to approve or reject transactions,
So that I can control or guide the user's spending.

6. Feature Flow

6.1 Invite Saver

1. User navigates to "Invite Saver"
 2. User enters email
 3. Select role:
 - Viewer
 - Approver
 4. Invite sent
 5. Saver accepts invitation
 6. Relationship becomes active
-

6.2 Viewer Mode Flow (Accountability)

1. User performs transaction
2. Transaction succeeds normally
3. Saver receives notification:

"John spent ₦5,000 on Food budget"

No impact on transaction execution.

6.3 Approval Mode Flow (Strict Control)

1. User initiates transaction
2. System checks → approval required
3. Transaction created with:

status = pending_approval

4. Approval request created
5. Notification sent to saver:

"John wants to spend ₦5,000 on Food budget"

6. Saver chooses:
 - Approve
 - Reject

6.4 On Approval

- Transaction status → success
- Wallet debited
- Ledger updated
- Reports updated

6.5 On Rejection

- Transaction status → rejected
- No wallet movement
- No ledger update

7. Functional Requirements

ID Requirement

FR1 User can invite saver

FR2 User can assign role (viewer / approver)

FR3 System stores relationship

FR4 System sends notifications on spend

FR5 System enforces approval when required

FR6 Transaction status supports pending approval

FR7 Approver can approve/reject

FR8 Ledger updates only after approval

FR9 History reflects approval status

8. Data Model

8.1 Saver Relationships

saver_relationships

- id
 - user_id
 - saver_id
 - role_type (viewer / approver)
 - status
 - created_at
-

8.2 Approvals

approvals

- id
 - transaction_id
 - requester_id
 - approver_id
 - status (pending / approved / rejected)
 - created_at
 - responded_at
-

8.3 Transaction Update

Add new statuses:

pending_approval

approved

rejected

9. Ledger Rules

- No ledger entry created until approval

- On approval → normal ledger flow
 - On rejection → no ledger entry
-

10. Notification Requirements

Viewer Mode

- Send notification after transaction

Approval Mode

- Send approval request notification
- Send approval result notification

Channels:

- In-app
 - Push
 - Email (optional)
-

11. Validation Rules

Before transaction:

- Check if user has approver
- Check role type

Before approval:

- Check transaction still pending
 - Check balance (optional re-check)
 - Check not expired
-

12. Edge Cases

- Approver offline
 - No response
 - Multiple requests
 - Insufficient balance after delay
 - Transaction expires
 - User cancels request
-

13. Acceptance Criteria

- User can invite saver
 - Role assignment works
 - Viewer receives notifications
 - Approver receives request
 - Transaction waits for approval
 - Approval executes transaction
 - Rejection cancels transaction
 - Ledger updates correctly
 - History reflects correct state
-

14. Future Enhancements

- Multiple approvers
- Approval limits (only above X amount)
- Time-based approval
- AI-assisted approval
- Group approval (family / business)
- Spending rules (auto approve below threshold)

15. Product Impact

This feature enables:

- Financial discipline
- Shared accountability
- Controlled spending
- Family finance use cases
- SME approval workflows